

LINE MCP Server

Getting Started Guide

LINE Development Team

September 13, 2026

1 What is LINE MCP?

LINE exposes its queueing analysis capabilities as a **Model Context Protocol (MCP)** server, allowing Large Language Models such as Claude to build, solve, and analyze queueing models conversationally. Instead of writing code manually, you describe your system in natural language and the LLM calls LINE tools to produce quantitative results.

The MCP server provides 9 tools ranging from quick one-line analyses (no code required) to full custom model building via Python code execution.

2 Setup

2.1 Installation

Install the LINE solver Python package. The MCP server and the MCP SDK are part of it, and the installation puts a `line-mcp` command on the path:

```
pip install line-solver
```

Alternatively, clone the repository:

```
git clone https://github.com/imperial-qore/line-solver.git
cd line-solver/python
pip install -e .
```

2.2 Claude Desktop Configuration

Add the following to your Claude Desktop configuration file (`claude_desktop_config.json`):

```
{
  "mcpServers": {
    "line-solver": {
      "command": "line-mcp"
    }
  }
}
```

If `line-mcp` is not on the path the MCP client sees, give its absolute location instead, the one which `line-mcp` prints. Running from a source checkout without installing the package also works: set `"command"` to `"python"` and `"args"` to `["/path/to/line-solver/python/mcp_server.py"]`.

2.3 Claude Code Configuration

For Claude Code (CLI) the shortest route is the command line:

```
claude mcp add line-solver -- line-mcp
```

If `line-mcp` is not on the path Claude Code sees, which is the usual case when the package was installed into a virtual environment, give the executable by absolute path and state the scope explicitly:

```
claude mcp add line-solver -s local -- [YOUR_PATH]/line-mcp/.venv/bin/line-mcp
```

Here `-s local` keeps the entry private to you in the current project; `-s user` would make it available in every project, and `-s project` would write it into the `.mcp.json` below.

To share the server with everyone working on a project, declare it instead in a `.mcp.json` file at the project root:

```
{
  "mcpServers": {
    "line-solver": {
      "type": "stdio",
      "command": "line-mcp"
    }
  }
}
```

Claude Code does *not* read `mcpServers` from `.claude/settings.json`; a server declared there is ignored without a warning. Run `claude mcp list` to confirm the server was picked up. A project-scoped server out of `.mcp.json` is listed as pending approval until it has been approved once in an interactive session.

Tip

Once configured, you can ask Claude questions like “What is the average queue length for an M/M/1 queue with arrival rate 2 and service rate 3?” and it will automatically use the LINE MCP tools to compute the answer.

3 Quick-Start Tools

These tools require no code—just provide parameter values.

3.1 analyze_queue — Open Queue Analysis

analyze_queue

Builds and solves an M/M/k (or M/M/k/N) open queueing model.

Parameter	Default	Description
arrival_rate	(required)	Job arrival rate (λ)
service_rate	(required)	Service rate per server (μ)
servers	1	Number of servers (k)
queue_capacity	-1	Total capacity (-1 = infinite)
scheduling	FCFS	FCFS, PS, LCFS, or INF
solver	MVA	MVA, CTMC, NC, SSA, FLD, or MAM
format	text	Output format: text or json

Example prompt: “Analyze an M/M/1 queue with arrival rate 2 and service rate 5.”

The LLM will call:

```
analyze_queue(arrival_rate=2.0, service_rate=5.0)
```

Example output:

```
Model: M/M/1
arrival_rate (lambda) = 2.0
service_rate (mu) = 5.0
servers (k) = 1
utilization (rho) = 0.400000
solver = MVA

Station Class QLen Tput RespT ResidT ArvR Util
Queue Class1 0.667 2.0 0.333 0.333 2.0 0.4
```

3.2 analyze_closed_network — Closed Network Analysis

analyze_closed_network

Builds and solves a closed queueing network with a Delay (think time) and a Queue.

Parameter	Default	Description
population	(required)	Number of circulating jobs (N)
think_time	(required)	Mean think time at Delay node (Z)
service_rate	(required)	Service rate per server (μ)
servers	1	Number of servers (k)
scheduling	FCFS	FCFS, PS, LCFS, or INF
solver	MVA	MVA, CTMC, NC, SSA, FLD, or MAM
format	text	Output format: text or json

Example prompt: “Analyze a closed system with 10 users, 5 seconds think time, and service rate 2.”

```
analyze_closed_network(population=10, think_time=5.0,
```

```
service_rate=2.0)
```

3.3 sweep_parameter — Parameter Sweep

sweep_parameter

Sweeps one parameter over a range and collects performance metrics at each point.

Parameter	Default	Description
param	(required)	Parameter to sweep (arrival_rate, service_rate, servers, population, think_time)
values	(required)	Comma-separated values or range notation (start:stop:step)
model_type	open	open or closed
<i>Plus all base parameters from analyze_queue/analyze_closed_network</i>		

Example prompt: “How does queue length change as arrival rate goes from 0.5 to 4.5 in steps of 0.5, with service rate 5?”

```
sweep_parameter(param="arrival_rate",
                values="0.5:4.5:0.5",
                service_rate=5.0)
```

4 Building Models Without Code

The quick-start tools above each build one fixed shape. These build any shape, still without writing Python. Each returns a `model_id`; pass that to `solve_model_file` for any analysis, or to `find_solver`, `visualize_model`, `compare_solvers` or `convert_model`.

4.1 build_network — Any Network From a Description

build_network

Builds an open, closed or mixed network with any number of stations and classes.

Parameter	Default	Description
stations	(required)	JSON list of stations, in routing order. Each has name and type (Source, Sink, Queue, Delay, Router, ClassSwitch, Fork, Join); optional scheduling, servers, capacity
classes	(required)	JSON list of classes. Each has name and type (Open or Closed); a closed class also needs population and refstation
service	—	JSON map station → class → distribution
arrival	—	JSON map for open classes at a Source
routing	serial	"serial", or a JSON list of hops [from, to, class, prob]
name	Model	Model name
format	text	text or json

Example prompt: “Model a web tier and a database serving interactive requests, with a batch job circulating alongside them.”

```

build_network(
  stations=' [{"name":"Src","type":"Source"}, '
            ' {"name":"Q1","type":"Queue","scheduling":"PS"}, '
            ' {"name":"Q2","type":"Queue"}, '
            ' {"name":"Snk","type":"Sink"}, '
            ' {"name":"Think","type":"Delay"} ] ',
  classes=' [{"name":"web","type":"Open"}, '
            ' {"name":"batch","type":"Closed"}, '
            ' {"population":4,"refstation":"Think"} ] ',
  service=' {"Q1":{"web":"exp(4.0)","batch":"exp(2.0)"}, '
            ' {"Q2":{"web":"exp(3.0)","batch":"exp(2.5)"}, '
            ' {"Think":{"batch":"exp(0.5)"} } ',
  arrival=' {"web":"exp(1.0)"} ',
  routing=' [ ["Src","Q1","web",1.0], ["Q1","Q2","web",1.0], '
            ' ["Q2","Snk","web",1.0], ["Think","Q1","batch",1.0], '
            ' ["Q1","Q2","batch",1.0], ["Q2","Think","batch",1.0]] ')

```

Open and closed classes may appear in the same model; that is what makes it mixed, and it is the case neither quick-start tool can express.

Distributions are written `name(args)` — `exp(2.0)`, `erlang(3.0, 2)`, `hyperexp(0.5, 1.0, 4.0)`, `det(0.25)`, `cox2(1.0, 2.0, 0.5)`, `gamma(2.0, 0.5)`, `replayer('trace.txt')`. The name is matched against LINE's own distribution classes, so every distribution the library publishes is available. A bare number is refused: a rate and a mean are not the same thing.

Caches, Petri-net places and transitions, finite-capacity regions and layered models are not built here — reach those with `build_from_gallery`, or hand a LINE json document to `solve_model_file`.

4.2 build_topology — From a Demand Matrix

build_topology

Builds a multi-station network from service demands; the topology supplies the routing. `kind` is `tandem`, `cyclic`, `cluster`, `cluster_closed` or `cluster_mixed`.

This is the textbook (N, Z, D) input, and the shortest route to a mixed network:

```

build_topology(kind="cluster_mixed",
  demands=' [[0.4, 0.5], [0.3, 0.2]] ',
  population=' [3] ', think_time=' [2.0] ',
  arrival_rates=' [1.0] ', scheduling="PS")

```

4.3 build_from_gallery and list_gallery

`list_gallery` catalogues the 71 ready-made models in `line_solver.gallery`, each with the arguments it takes. Between them they cover fork-join, caches, layered networks, finite-capacity regions, random environments and the classical arrival/service matrix.

```

list_gallery()
build_from_gallery(name="cqn", params=' {"M": 3} ')
build_from_gallery(name="fj_open")

```

4.4 sweep_model — Sweep Any Field

Where `sweep_parameter` varies one of five preset knobs of the two preset shapes, `sweep_model` varies any field of any builder's arguments, named by a dotted path.

```
sweep_model(param="population.0", values="2:20:2",
            builder="topology",
            args='{"kind":"cyclic", "demands": [[0.5], [0.2]], '
                '"population": [2]}')
```

A field holding an integer is swept with integers. For a field that is not numeric — a service distribution, say — give values as a JSON list:

```
sweep_model(param="service.Q.c",
            values='["exp(2.0)", "exp(4.0)", "exp(8.0)"]',
            builder="network", args='{...}')
```

4.5 convert_model — Export and Interchange

Converts a model from any source (`model_id`, `file`, `content` or `code`) to `json` (the LINE interchange document), `jsimg` (JMT), `pnml`, `lqnx` or `python` (the equivalent script). Give `output_file` to write it rather than return it.

```
convert_model(target="json", model_id="m1")
convert_model(target="python", model_id="m1", name="my_model")
```

The `json` document round-trips: hand it straight back to `solve_model_file` as `content` and it solves identically.

5 Building Custom Models

The `solve_line_model` tool executes arbitrary LINE Python code in a sandboxed environment with `from line_solver import *`, `numpy`, and `pandas` pre-loaded.

5.1 Open Network Example

```
model = Network('M/M/1')
source = Source(model, 'Source')
queue = Queue(model, 'Queue', SchedStrategy.FCFS)
sink = Sink(model, 'Sink')

oclass = OpenClass(model, 'Class1')
source.setArrival(oclass, Exp(1.0))
queue.setService(oclass, Exp(2.0))

model.link(Network.serial_routing([source, queue, sink]))
avg_table = MVA(model).avg_table()
print(avg_table)
```

5.2 Closed Network Example

```
model = Network('ClosedQN')
delay = Delay(model, 'Think')
queue = Queue(model, 'Queue', SchedStrategy.PS)

cclass = ClosedClass(model, 'Users', 5, delay)
delay.setService(cclass, Exp(1.0))
queue.setService(cclass, Exp(3.0))

model.link(Network.serial_routing([delay, queue]))
avg_table = MVA(model).avg_table()
print(avg_table)
```

5.3 Multiclass Example

```
model = Network('Multiclass')
source = Source(model, 'Source')
queue = Queue(model, 'Queue', SchedStrategy.FCFS)
sink = Sink(model, 'Sink')

class1 = OpenClass(model, 'Web')
class2 = OpenClass(model, 'API')

source.setArrival(class1, Exp(1.0))
source.setArrival(class2, Exp(0.5))
queue.setService(class1, Exp(4.0))
queue.setService(class2, Exp(3.0))

model.link(Network.serial_routing([source, queue, sink]))
avg_table = MVA(model).avg_table()
print(avg_table)
```

5.4 Persistent Sessions

Use `session_id` to persist variables across multiple `solve_line_model` calls:

```
# Call 1: Build model
solve_line_model(code="...", session_id="my_session")

# Call 2: Reuse model from previous call
solve_line_model(
    code="avg_table = NC(model).avg_table()\nprint(avg_table)",
    session_id="my_session"
)
```

6 Advanced Tools

6.1 compare_solvers — Multi-Solver Comparison

Runs the same model with multiple solvers and returns a merged results table.

```
compare_solvers(
    code="""
model = Network('M/M/1')
source = Source(model, 'Source')
queue = Queue(model, 'Queue', SchedStrategy.FCFS)
sink = Sink(model, 'Sink')
oclass = OpenClass(model, 'Class1')
source.setArrival(oclass, Exp(1.0))
queue.setService(oclass, Exp(2.0))
model.link(Network.serial_routing([source, queue, sink]))
""",
    solvers="MVA,NC,MAM"
)
```

6.2 visualize_model — Mermaid Diagrams

Generates a Mermaid graph definition from a queueing network model, showing nodes, connections, and class information.

```
visualize_model(code="""
model = Network('WebApp')
source = Source(model, 'Arrivals')
lb = Queue(model, 'LoadBalancer', SchedStrategy.PS)
app = Queue(model, 'AppServer', SchedStrategy.FCFS)
sink = Sink(model, 'Departures')
oclass = OpenClass(model, 'Requests')
source.setArrival(oclass, Exp(5.0))
lb.setService(oclass, Exp(100.0))
app.setService(oclass, Exp(10.0))
P = model.initRoutingMatrix()
P.set(oclass, oclass, source, lb, 1.0)
P.set(oclass, oclass, lb, app, 1.0)
P.set(oclass, oclass, app, sink, 1.0)
model.link(P)
""")
```

which returns:

```
graph LR
    N0(["Arrivals"])
    N1["LoadBalancer"]
    N2["AppServer"]
    N3(["Departures"])
    N0 --> N1
    N1 --> N2
    N2 --> N3
    style N1,N2 fill:#2196F3,color:#fff,stroke:#2196F3
```

```
style N3 fill:#F44336,color:#fff,stroke:#F44336
style N0 fill:#4CAF50,color:#fff,stroke:#4CAF50

%% Classes: Requests (open)
```

The output is a Mermaid diagram that can be rendered by any Mermaid-compatible tool (GitHub, Notion, Mermaid Live Editor, etc.).

Node shape carries node type: a stadium is a Source or Sink, a rectangle a Queue or ClassSwitch, a rounded box a Delay or Router, a hexagon a Fork or Join, a cylinder a Cache. A station label picks up its server count, either ($k=N$) for a multiserver or (IS) for an infinite server, and the trailing comment line lists every class with its population, (*open*) or ($N=...$). Labels are always quoted, since a bare Mermaid label may not hold parentheses.

Routing is declared one hop at a time with `P.set(from_class, to_class, from_node, to_node, prob)`, or a whole class at a time by assigning a node-by-node matrix to `P[jobclass, jobclass]`. Indexing a routing matrix by node, as in `P[jobclass, src, dst] = p`, is not supported and raises `ValueError`. For a plain chain, `Network.serial_routing([...])` builds the matrix for you.

6.3 list_examples and get_example_code

Browse and retrieve source code from LINE's built-in example gallery:

- `list_examples()` — returns a categorized list of available examples
- `get_example_code(name)` — returns the Python source code for a named example (e.g., "gallery_mm1", "cqn_multiserver")

Example prompt: “Show me the code for the multiclass open queueing network example.”

6.4 clear_session — Session Cleanup

Removes a persistent code session created with `session_id`:

```
clear_session(session_id="my_session")
```

7 Solving Model Files

The tools so far build a model out of Python code. A command line does not: it reads a *document*, picks a solver by method name, asks for one of the analyses the library publishes and renders the answer. `solve_model_file` is that surface, so a client holding a model file need not paste it into code, and every analysis LINE publishes is reachable without writing the getter call by hand. It is the `line-cli` command line expressed as one tool, and it accepts the same vocabulary.

7.1 solve_model_file — Solve a Model Document

```
solve_model_file(file="model.json", solver="mva", analysis="avg")
solve_model_file(file="webapp.jsimg", solver="ctmc", analysis="prob", node
=1, state="2")
```

```
solve_model_file(file="database.lqnx", solver="ln", analysis="avg", output="
    json")
solve_model_file(content=document_text, solver="nc", analysis="normconst")
```

The model comes either as file (a path) or as content (the document itself). Its format is taken from the extension, from the first bytes of the text, or from an explicit `input_format`:

- `json` — LINE's own interchange model, carrying a Network, a LayeredNetwork or an Environment
- `jsim`, `jsimg`, `jsimw` — JMT simulation documents
- `lqnx`, `xml` — layered queueing networks
- `pnml` — place/transition nets (ISO/IEC 15909-2)
- `mat`, `pkl` — MATLAB and pickled models, by path only

The `analysis` argument takes one name or a comma-separated list, and covers the average, system, chain, node and node-chain tables; the cache and per-item tables; the retrial orbit, the two loss views and the deadline-miss table; the normalizing constant and the subnetwork busy period; response-time and passage-time distributions and percentiles; the transient tables; the state probabilities, marginal and joint; simulated sample paths; rewards; sensitivities to the service demands; and the SolverUQ interval. Several of these need a particular solver, and `list_solvers` reports which.

Every numeric control the command line carries is an argument here: `method`, `seed`, `samples`, `cutoff`, `timespan`, `timestep`, `tol`, `iter_tol`, `iter_max`, `multiserver`, `fork_join`, `warmupfrac`, `stage_solver`, `uq_solver`, the `busyperiod` pair and the three `sens` controls. Anything else SolverOptions understands may be passed as a JSON object in `options`.

7.2 find_solver — Which Solver Fits This Model

Reports rather than solves. One row per (solver, method) pair, saying whether it runs, whether it is exact, approximate, a bound or a simulation, and which measures it answers:

```
find_solver(file="model.json")
find_solver(file="model.json", metric="cdf")
find_solver(file="model.json", show_all=True)
find_solver(code="model = Network('M/M/1') ...")
```

`metric` narrows the report to a measure group (`cdf`, `tran`, `prob`, `sample`) or to an accessor (`getCdfRespT`); `show_all` keeps the pairs that are refused, with the reason each was refused. The `Method` column is the method name to hand straight back to `solve_model_file`.

7.3 list_solvers, check_install and get_version

- `list_solvers()` — the vocabulary: solver method names and their aliases, every analysis, which analyses require which solver, which solvers each input format admits, and the output formats
- `check_install()` — the environment check, reporting on the Java runtime the JMT and LDES solvers need and on the SageMath symbolic backend
- `get_version()` — the LINE release, the interpreter, and where the examples are being read from

8 Solver Reference

Solver	Best For
MVA	Product-form networks with exponential service (fast, exact)
NC	Closed networks needing exact normalizing-constant analysis
CTMC	Small state-space models; supports non-product-form features
MAM	Phase-type / MAP service distributions (matrix analytic)
SSA	Stochastic simulation; works for any supported model
FLD	Fluid / ODE approximation for transient and large models

Table 1: LINE MCP solver guide

All solvers are available through every MCP tool. The quick-start tools (`analyze_queue`, `analyze_closed_network`) accept a `solver` parameter; for custom models via `solve_line_model`, instantiate the solver class directly (e.g., `MVA(model)`, `CTMC(model)`).

9 Output Formats

All analysis tools accept a `format` parameter:

- **text** (default) — Human-readable table output with model summary header.
- **json** — Structured JSON with `metadata` and `results` keys, suitable for programmatic consumption.

A run that never reaches a result answers in the requested format too: in `json` mode a rejected input, an unstable system or a solver failure comes back as `{"error": "..."}` rather than as plain text, so a caller that parses the reply reads the reason instead of a parse failure.

JSON output example:

```
{
  "metadata": {
    "model": "M/M/1",
    "arrival_rate": 2.0,
    "service_rate": 5.0,
    "utilization": 0.4,
    "solver": "MVA"
  },
  "results": [
    {
      "Station": "Queue",
      "Class": "Class1",
      "QLen": 0.6667,
      "Tput": 2.0,
      "RespT": 0.3333,
      "Util": 0.4
    }
  ]
}
```

10 Tool Summary

Tool	Purpose
<code>build_network</code>	Build ANY network from a description: open, closed or mixed, many stations, many classes (no code)
<code>build_topology</code>	Build a multi-station network from a demand matrix, with no routing
<code>build_from_gallery</code>	Build one of the 71 ready-made gallery models
<code>list_gallery</code>	The gallery catalogue, with each model's arguments
<code>convert_model</code>	Convert a model to LINE json, JMT, PNML, .lqnx or Python
<code>sweep_model</code>	Sweep any field of any model description
<code>analyze_queue</code>	Quick M/M/k or M/M/k/N open-queue analysis (no code)
<code>analyze_closed_network</code>	Quick closed-network analysis (no code)
<code>sweep_parameter</code>	Sweep one parameter over a range and collect metrics
<code>solve_model_file</code>	Solve a model document: the whole <code>line-cli</code> command line
<code>find_solver</code>	Which solvers and methods can analyze a model, and why not
<code>list_solvers</code>	The vocabulary: solvers, analyses, input and output formats
<code>solve_line_model</code>	Execute arbitrary LINE Python code (sandboxed)
<code>compare_solvers</code>	Run the same model with multiple solvers side-by-side
<code>visualize_model</code>	Generate a Mermaid diagram of a queueing network
<code>list_examples</code>	Browse available example models by category
<code>get_example_code</code>	Retrieve source code of a named example
<code>check_install</code>	Environment check: which optional backends are reachable
<code>get_version</code>	LINE version, interpreter and examples directory
<code>clear_session</code>	Remove a persistent code session

Table 2: All LINE MCP tools